More on Classes and Objects

Dice, methods, and Yahtzee

TANGENT!

f-strings

F-STRINGS

- Putting an `f` right before the opening quote makes an **f-string**
- Inside an f-string, anything in `{ }` is run as Python, and its value goes into the string
- No `+` and no `str()` needed

```
name = "Finn"
age = 14
print("Hi " + name + ", you are " + str(age))
print(f"Hi {name}, you are {age}")
```

```
Hi Finn, you are 14
Hi Finn, you are 14
```

ANYTHING CAN GO IN THE BRACES

- The braces can hold any expression: a variable, math, a function call
- Text outside the braces is copied as is
- Forget the `f` and the braces are just characters

```
x = 3
print(f"{x} + 1 = {x + 1}")
print(f"[{x}]")
print("[{x}]")
print(f"{len('hello')} letters")
```

Predict: what does each line print?

ANYTHING CAN GO IN THE BRACES

- The braces can hold any expression: a variable, math, a function call
- Text outside the braces is copied as is
- Forget the `f` and the braces are just characters

```
x = 3
print(f"{x} + 1 = {x + 1}")
print(f"[{x}]")
print("[{x}]")
print(f"{len('hello')} letters")
```

```
3 + 1 = 4
[3]
[{x}]
5 letters
```

TANGENT OVER!

back to classes and objects

OBJECTS YOU ALREADY USE

- Strings and lists are **objects**
- The functions you call with a dot, like `upper` and `append`, are called **methods**
- A method belongs to its object's type: lists have `append`, strings don't

```
s = "hello"
s.upper()
print(s)

names = ["Finn"]
names.append("Jake")
print(names)
```

Predict: what do the two `print` lines show?

OBJECTS YOU ALREADY USE

- Strings and lists are **objects**
- The functions you call with a dot, like `upper` and `append`, are called **methods**
- A method belongs to its object's type: lists have `append`, strings don't

```
s = "hello"
s.upper()
print(s)

names = ["Finn"]
names.append("Jake")
print(names)
```

```
hello
['Finn', 'Jake']
```

upper gives back a new string; s doesn't change
append changes the list itself

str AND list ARE CLASSES

- `str` and `list` are classes that come with Python
- Somebody wrote `upper` and `append` inside those classes
- Last time we made our own class, `Time`, but we wrote its functions outside the class
- Today our class gets its own methods, the way `str` and `list` have

```
print(type("hello"))
print(type(["Finn"]))
```

```
<class 'str'>
<class 'list'>
```

DICE (SINGULAR)

- `Die` is a class for one six-sided die
- For now we only use it; we'll look inside on the next slide
- (though the whole class is visible on GitHub if you want to preview)
- `Die(3)` makes a new die showing 3
- `roll` is a method: it gives the die a new random value from 1 to 6

```
d = Die(3)
print(d.value)
d.roll()
print(d.value)
print(d)
```

Predict: what does the last line print?

DICE (SINGULAR)

- `Die` is a class for one six-sided die
- For now we only use it; we'll look inside on the next slide
- `Die(3)` makes a new die showing 3
- `roll` is a method: it gives the die a new random value from 1 to 6

```
d = Die(3)
print(d.value)
d.roll()
print(d.value)
print(d)
```

```
3
5
<__main__.Die object at 0x7f84e78ea510>
```

line 2 is random, so yours may be different
your address (0x...) will be different too

Python doesn't know how to show a die yet

INSIDE THE DIE

- A `def` inside a class makes a **method**
- Every method's first parameter is `self`
- `__init__` runs when you make a new `Die`
- `random.randint(1, 6)` gives a random int from 1 to 6, including both

```
import random

class Die:
    """One six-sided die."""

    def __init__(self, value):
        self.value = value

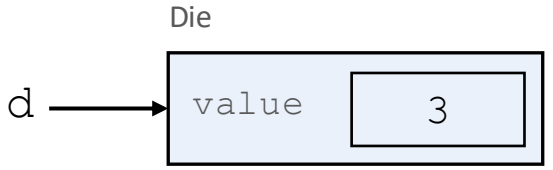
    def roll(self):
        self.value = random.randint(1, 6)
```

WHAT DIE(3) DOES

1. Python makes a new, empty `Die` object
2. It calls `__init__`, with `self` set to the new object and `value` set to `3`
3. Inside `__init__`, `self.value = value` stores the 3 in the object
4. Then `Die(3)` gives back the new object, and `d` refers to it

```
d = Die(3)
print(d.value)
```

```
3
```



__init__ REPLACES MAKE_TIME

- Same job: fill in the new object's attributes
- With `__init__`, Python makes the object and gives it back for you
- Every `Time` gets `hour`, `minute`, and `second`, and the class is the one place that says so

last time

```
class Time:
    """A time of day."""

def make_time(hour, minute, second):
    time = Time()
    time.hour = hour
    time.minute = minute
    time.second = second
    return time

start = make_time(9, 20, 0)
print(start.minute)
```

```
20
```

today

```
class Time:
    """A time of day."""

    def __init__(self, hour, minute, second):
        self.hour = hour
        self.minute = minute
        self.second = second

start = Time(9, 20, 0)
print(start.minute)
```

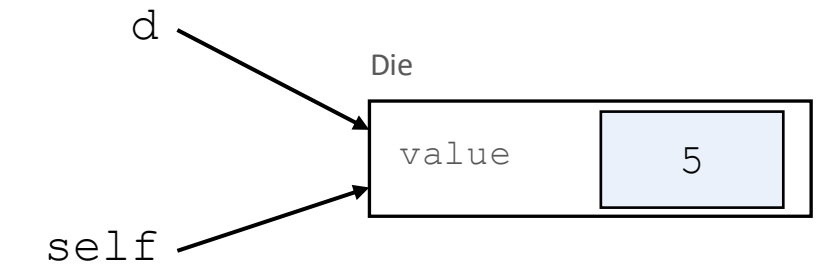
```
20
```

WHERE self COMES FROM

- `d.roll()` runs `roll` with `self` set to `d`
- You could write the same call the long way: `Die.roll(d)`
- Nobody writes it the long way. It just shows where `self` comes from
- Inside `roll`, `self` and `d` are two names for the same die

```
d = Die(3)
Die.roll(d)     # same as d.roll()
print(d.value)
```

```
5
```



while roll runs

ERROR MESSAGES ABOUT self

```
d = Die(3)
d.roll(5)
```

```
TypeError: Die.roll() takes 1 positional
argument but 2 were given
```

You passed one argument. Python also passed `d` as `self`, so that's two.

```
class Die:
    ...

    def show():
        print(self.value)

d = Die(3)
d.show()
```

```
TypeError: Die.show() takes 0 positional
arguments but 1 was given
```

The method forgot `self` in its parameter list, but Python still passes `d`.

```
Die.roll()
```

```
TypeError: Die.roll() missing 1 required
positional argument: 'self'
```

Called on the class, with no die before the dot, there's nothing to use as `self`.

TWO DICE

- Each call to `Die(...)` makes a separate object
- Each object has its own `value`

```
a = Die(2)
b = Die(5)
a.roll()
print(b.value)
```

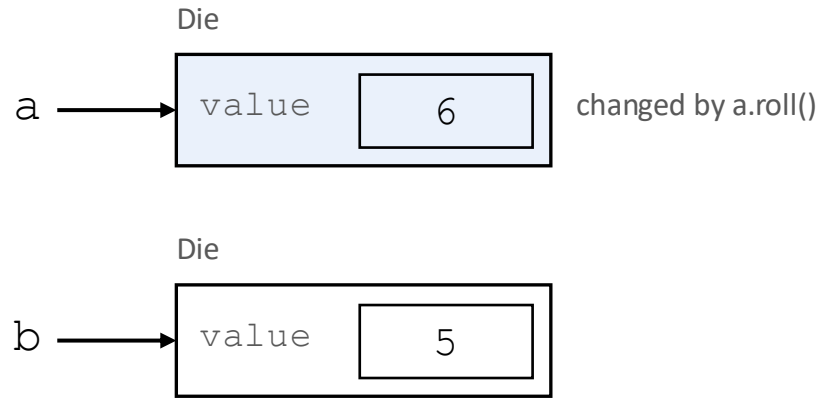
Predict: does `a.roll()` change `b`? What does the last line print?

TWO DICE

- Each call to `Die(...)` makes a separate object
- Each object has its own `value`
- `a.roll()` runs `roll` with `self` set to `a`, so only `a` changes

```
a = Die(2)
b = Die(5)
a.roll()
print(b.value)
```

```
5
```



YOUR TURN: A CARD CLASS

- A playing card has a rank (1 to 13) and a suit (`"hearts"`, `"spades"`, ...)
- Write a class `Card` whose `__init__` stores both
- Add a method `color`: `"red"` for hearts and diamonds, `"black"` otherwise
- **In-class sheet, problem 1**

```
c = Card(12, "hearts")
print(c.rank, c.suit)
print(c.color())
print(Card(5, "spades").color())
```

should print:

```
12 hearts
red
black
```

DUNDER METHODS

- Methods with two underscores on each side, like `__init__`, are special
- "Double underscore" is shortened to **dunder**: say `__init__` as "dunder init"
- You don't call them yourself. Python calls them when you use its own syntax and functions

When you write	Python calls	Today's job
`Die(3)`	`__init__`	make a die (done)
`print(d)`	`__str__`	show a die
`print(dice)`	`__repr__`	show a list of dice
`a == b`	`__eq__`	compare two dice
`sorted(dice),max(dice)`	`__lt__`	sort dice

TEACHING print ABOUT DICE

- `print(d)` calls `d.__str__()` and prints the string it gets back
- `__str__` must **return** a string. Don't `print` inside it
- f-strings and `str(d)` use `__str__` too

```
class Die:
    ...

    def __str__(self):
        return f"[{self.value}]"
```

```
d = Die(4)
print(d)
print(f"you rolled {d}")
```

```
[4]
you rolled [4]
```

PRINTING A list OF DICE

- `Die` has `__str__` now, so `print(d)` works
- What about a list of dice?

```
dice = [Die(5), Die(2)]
print(dice[0])
print(dice)
```

Predict: what do the two lines print?

PRINTING A list OF DICE

- `Die` has `__str__` now, so `print(d)` works
- What about a list of dice?

```
dice = [Die(5), Die(2)]
print(dice[0])
print(dice)
```

```
[5]
[<__main__.Die object at 0x7f84e78eae40>,
 <__main__.Die object at 0x7f84e7906210>]
```

(the last line is one line, wrapped to fit)

__repr__ IS FOR PROGRAMMERS

- A list shows its items with a different dunder method, `__repr__`
- `__str__` is for the person using your program
- `__repr__` is for the programmer. By convention it looks like the code that makes the object

```
class Die:
    ...

    def __repr__(self):
        return f"Die({self.value})"
```

```
dice = [Die(5), Die(2)]
print(dice[0])
print(dice)
```

```
[5]
[Die(5), Die(2)]
```

YOUR TURN: __str__ AND __repr__

- Add `__str__` and `__repr__` to your `Card`
- `__str__` returns something like `"12 of hearts"`
- `__repr__` returns the code that would make the card: `"Card(12, 'hearts')"`
- **In-class sheet, problem 2**

```
c = Card(12, "hearts")
print(c)
print([c, Card(1, "spades")])
print("you drew the " + str(c))
```

should print:

```
12 of hearts
[Card(12, 'hearts'), Card(1, 'spades')]
you drew the 12 of hearts
```

TWO DICE THAT MATCH

- Two dice, both showing 4
- `is` asks: are these the same object?
- `==` asks: are these equal?

```
a = Die(4)
b = Die(4)
print(a == b)
print(a is b)
```

Predict: what do the last two lines print?

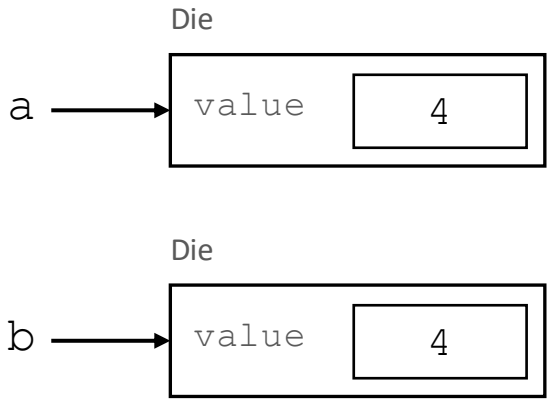
TWO DICE THAT MATCH

- Two dice, both showing 4
- `is` asks: are these the same object?
- `==` asks: are these equal?

```
a = Die(4)
b = Die(4)
print(a == b)
print(a is b)
```

```
False
False
```

For a new class, == does the same thing as is.
Python doesn't know what "equal" means for dice.



TEACHING == ABOUT DICE

- `a == b` calls `a.__eq__(b)`: `self` is `a`, `other` is `b`
- We decide what "equal" means: same value
- `!=` works now too
- Your dates assignment had the same problem: `same_date(x, y)` was `True` while `x == y` was `False`

```
class Die:
    ...

    def __eq__(self, other):
        return self.value == other.value
```

```
a = Die(4)
b = Die(4)
print(a == b)
print(a is b)
print(a != Die(6))
```

```
True
False
True
```

YOUR TURN: __EQ__

- Two cards are equal when the rank **and** the suit match
- Predict the three lines **before** you add `__eq__`
- Then add `__eq__` to your `Card` and run it again
- **In-class sheet, problem 3**

```
a = Card(12, "hearts")
print(a == Card(12, "hearts"))
print(a == Card(12, "spades"))

pile = [Card(1, "clubs"), Card(5, "clubs")]
print(Card(5, "clubs") in pile)
```

SORTING DICE

- In Yahtzee, a straight is easier to spot when the dice are in order
- `sorted` gives back a new list, in order

```
dice = [Die(5), Die(2), Die(6)]
print(sorted(dice))
```

Predict: what does the last line print?

SORTING DICE

- In Yahtzee, a straight is easier to spot when the dice are in order
- `sorted` gives back a new list, in order

```
dice = [Die(5), Die(2), Die(6)]
print(sorted(dice))
```

```
TypeError: '<' not supported between
instances of 'Die' and 'Die'
```

To sort, Python compares items with <.
It doesn't know how to compare two dice.

TEACHING < ABOUT DICE

- `a < b` calls `a.__lt__(b)`; "lt" is short for "less than"
- `sorted`, `min`, and `max` all compare with `<`
- `>` works too: Python turns `a > b` into `b < a`
- `sorted` gives back a **new** list. `dice` doesn't change

```
class Die:
    ...

    def __lt__(self, other):
        return self.value < other.value
```

```
dice = [Die(5), Die(2), Die(6)]
print(sorted(dice))
print(max(dice))
print(dice)
```

```
[Die(2), Die(5), Die(6)]
[6]
[Die(5), Die(2), Die(6)]
```

YOUR TURN: SORTING CARDS

- Add `__lt__` to your `Card`: compare ranks only
- Then predict each line before you run it
- **In-class sheet, problem 4**

```
hand = [Card(12, "hearts"),
        Card(3, "spades"),
        Card(7, "clubs")]
print(sorted(hand))
print(max(hand))
print(Card(5, "hearts") < Card(5, "spades"))
print(Card(5, "spades") < Card(5, "hearts"))
```

WHAT WE TAUGHT PYTHON

- Five dunder methods, each one or two lines long
- Now a `Die` works with `print`, `==`, `in`, `sorted`, `min`, and `max`, the same way a string or a number does

When you write	Python calls	What our Die does
`Die(3)`	`__init__`	stores the value
`print(d)`, `str(d)`, f-strings	`__str__`	returns `"[3]"`
`print(dice)`	`__repr__`	returns `"Die(3)"`
`a == b`, `in`	`__eq__`	compares values
`a < b`, `sorted`, `min`, `max`	`__lt__`	compares values

QUESTIONS YOU MIGHT BE ASKING

Does it have to be called self?

No, it's only a name. Python passes the object as the first argument no matter what you call it. Everyone uses self, so you should too.

Can I write d == 4?

No. __eq__ reads other.value, and 4 has no .value, so you get AttributeError. Compare d.value == 4.

What about <= and >=?

Not defined, so they raise TypeError. We only need < for sorting.

Does __init__ stop typos like d.vaule = 3?

No. Python makes a new attribute, the same as birthday.dya in the dates assignment. __init__ is where we write down what a Die should have.

Do I need all five dunder methods?

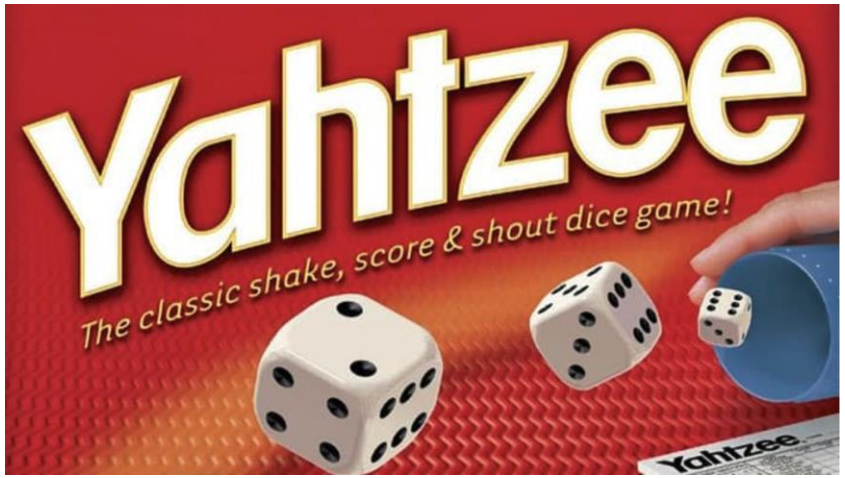
Only the ones you use. A class with only __init__ is fine; Python just can't print, compare, or sort it.

YAHTZEE

probably the most complex project this semester

YAHTZEE

- A game is 13 turns
- Each turn: roll five dice, then reroll any of them, up to two more times
- Then pick one of 13 categories to score the dice in. Each category gets used once
- If the dice don't fit the category you pick, you score 0 there



Category	Example	Points
threes	3 3 5 2 3	9
full house	2 5 2 5 5	25
small straight	3 4 5 3 6	30
large straight	5 3 2 4 6	40
yahtzee	6 6 6 6 6	50
chance	1 4 2 6 6	19

6 of the 13 categories; the handout has all of them

THREE CLASSES

- `Die`: one die, the class from Monday's slides
- `Hand`: the five dice you're rolling this turn
- `Scorecard`: your 13 scores
- Scoring rules, like `score_full_house(hand)`, are plain functions that take a `Hand`

`Die`

one die

attribute: value

```
roll()   __str__   __eq__   __lt__
...
```

`Hand`

five dice

attribute: dice

```
roll_all()   reroll()   counts()
...
```

`Scorecard`

13 scores

attribute: scores

```
record()   available()   total()
...
```

A HAND HOLDS DICE

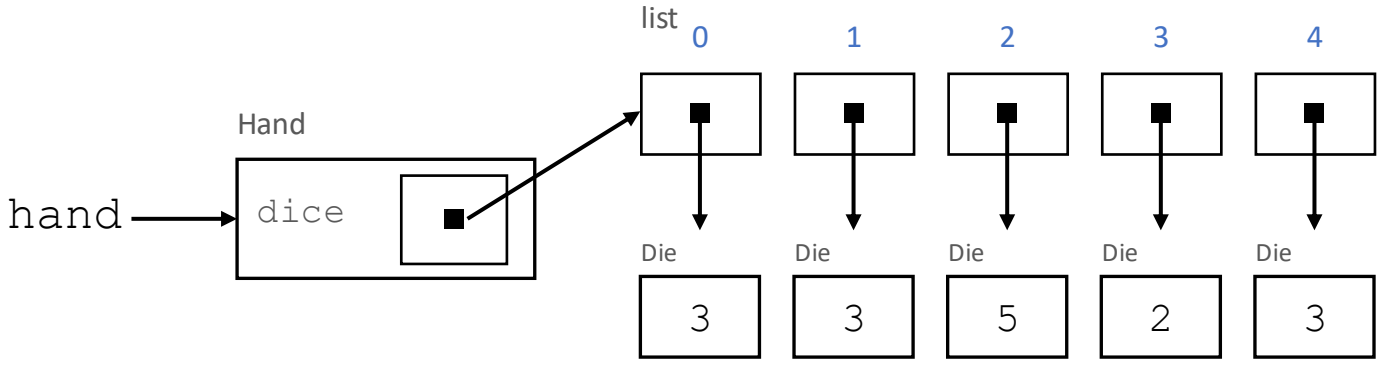
- `Hand([3, 3, 5, 2, 3])` makes five `Die` objects and keeps them in a list, `self.dice`
- Choosing the values makes testing easy. In the real game we make a hand, then roll it
- This is item 1 in the handout's Stage 2

```
class Hand:
    """Five dice."""

    def __init__(self, values):
        self.dice = []
        for v in values:
            self.dice.append(Die(v))
```

```
hand = Hand([3, 3, 5, 2, 3])
print(hand.dice)
```

```
[Die(3), Die(3), Die(5), Die(2), Die(3)]
```



READ METHOD CALLS LEFT TO RIGHT

- Read a long expression left to right, one step at a time
- `hand.dice` is a list; `hand.dice[2]` is a `Die`; `hand.dice[2].value` is an int

```
hand = Hand([3, 3, 5, 2, 3])
print(hand.dice[2])
print(hand.dice[2].value)
print(hand.dice[2].value + 1)
```

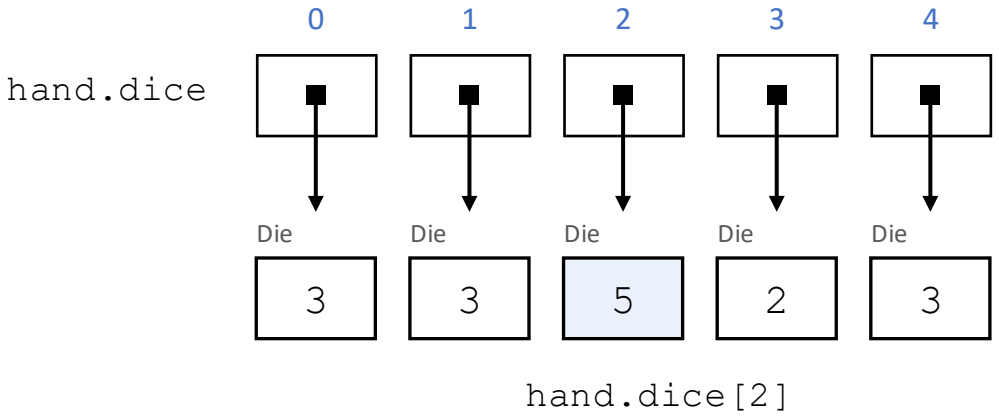
Predict: what do the three `print` lines show?

FOLLOWING THE DOTS

- Read a long expression left to right, one step at a time
- `hand.dice` is a list; `hand.dice[2]` is a `Die`; `hand.dice[2].value` is an int

```
hand = Hand([3, 3, 5, 2, 3])
print(hand.dice[2])
print(hand.dice[2].value)
print(hand.dice[2].value + 1)
```

```
[5]
5
6
```



LOOPING OVER THE DICE

- Inside a method, `self.dice` is this hand's list of dice
- `values` builds a plain list of ints, which is useful for testing
- Most `Hand` methods are this shape: loop over `self.dice`, look at each `die.value`

```
class Hand:
    ...

    def values(self):
        result = []
        for die in self.dice:
            result.append(die.value)
        return result
```

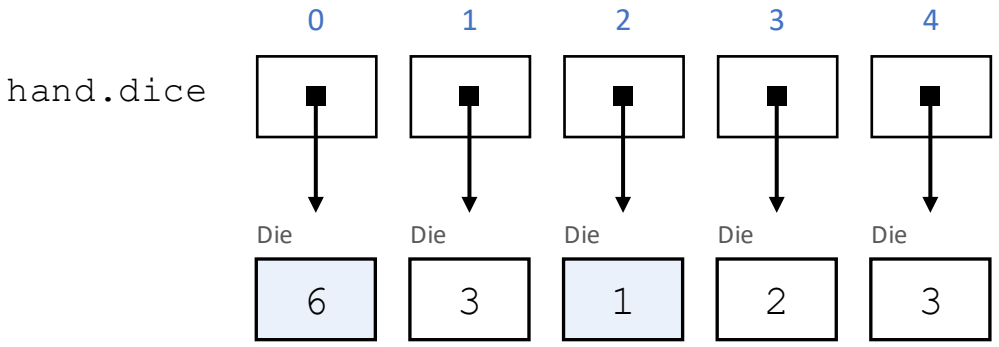
```
hand = Hand([3, 3, 5, 2, 3])
print(hand.values())
print(hand.total())
print(hand.count(3))
```

```
[3, 3, 5, 2, 3]
16
3
```

you'll write total and count

REROLLING DICE

- `reroll` takes a list of positions and rolls *only* those dice
- It changes the hand **in place** and returns nothing
- Players count from 1; Python counts from 0. `parse_positions("1 3")` gives `[0, 2]`
- `print(hand)` uses the `__str__` you'll write for `Hand`



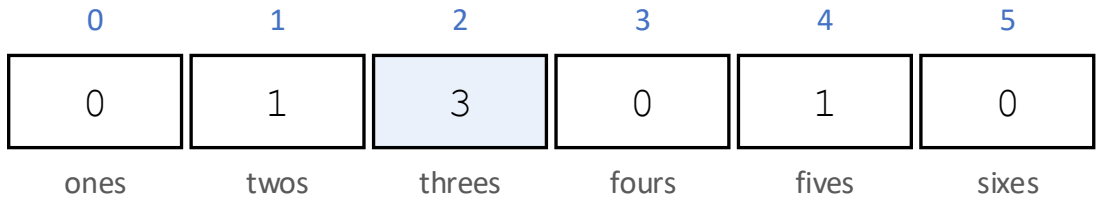

```
hand = Hand([3, 3, 5, 2, 3])
print(hand)
hand.reroll(parse_positions("1 3"))
print(hand)
```

```
1:3  2:3  3:5  4:2  5:3
1:6  2:3  3:1  4:2  5:3
```

dice 1 and 3 were rerolled; yours will be different

THE COUNTS LIST

- `counts()` returns six ints: how many ones, how many twos, ... how many sixes
- The count of threes is at index **2**
- Most scoring rules become one question about this list




```
hand = Hand([3, 3, 5, 2, 3])
print(hand.counts())
```

```
[0, 1, 3, 0, 1, 0]
```

`counts() is [0, 2, 0, 0, 3, 0]`

Predict: what five dice could make this?

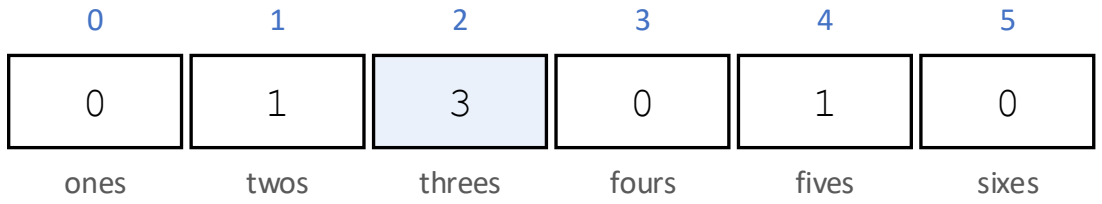
Predict: is it a full house? A three of a kind?

THE COUNTS LIST

- `counts()` returns six ints: how many ones, how many twos, ... how many sixes
- The count of threes is at index **2**
- Most scoring rules become one question about this list

```
hand = Hand([3, 3, 5, 2, 3])
print(hand.counts())
```

```
[0, 1, 3, 0, 1, 0]
```



`counts() is [0, 2, 0, 0, 3, 0]`

Two 2s and three 5s, like 2 5 2 5 5, in any order
Full house: yes, 3 and 2 are both in the list
Three of a kind: yes, some count is at least 3

SCORING RULES ARE FUNCTIONS

- Each scoring rule answers a question about a hand, like "is this a full house?"
- So the scoring rules are plain functions that take a `Hand`
- Not everything has to go inside a class
- Each one is short because `Hand` did the work

```
def score_chance(hand):
    return hand.total()
```

```
hand = Hand([1, 4, 2, 6, 6])
print(score_chance(hand))
print(score_for(hand, "sixes"))
print(score_for(hand, "full house"))
```

```
19
12
0
```

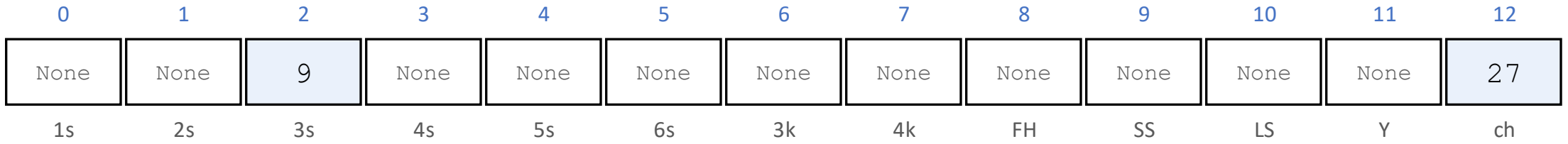
THE SCORECARD

- `self.scores` is a list of 13 scores, in the same order as `CATEGORIES`
- `None` means "not used yet"; `0` means "used, and scored nothing"
- `record` scores a hand in one category and stores it in place

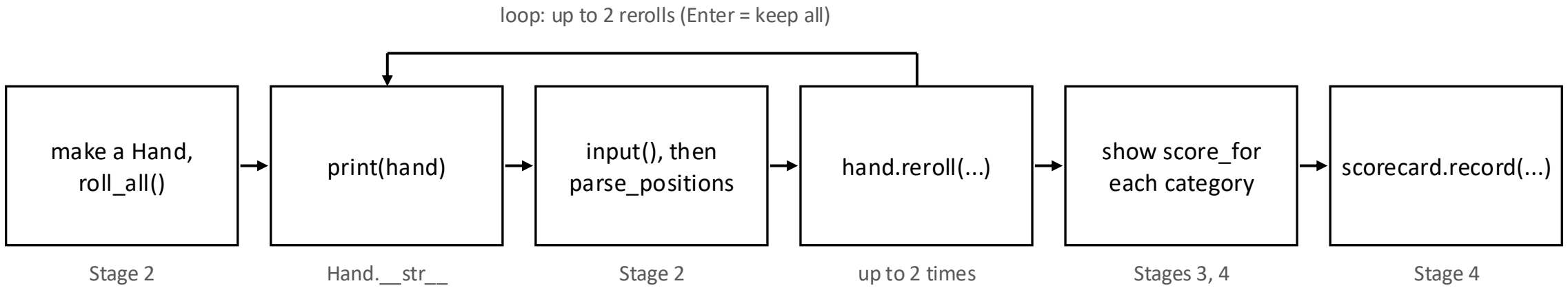
```
card = Scorecard()
card.record("threes", Hand([3, 3, 5, 2, 3]))
card.record("chance", Hand([6, 6, 5, 4, 6]))
print(card.scores[:3])
print(card.total())
```

```
[None, None, 9]
36
```

`card.scores`



ONE TURN



play_turn(scorecard) does these steps in order. play_game (provided) calls it 13 times with the same scorecard.

HOW TO WORK ON YAHTZEE

- Write one method, run its test, compare line by line, then move on
- Use the "Uses" column: big pieces call small ones
- Stage 1 is the `Die` from the slides; you built the same methods for `Card`

Stage	What you write	Pieces
1	`Die`	6 methods (from the slides)
2	`Hand`, `parse_positions`	10
3	scoring functions	8
4	`score_for`, `Scorecard`	7
5	`play_turn`	1, then play a game

QUESTIONS ABOUT YAHTZEE

Why does Hand take values instead of rolling itself?

So the tests can make the exact hand they need. In the game, make any hand and call roll_all().

Why does print(hand.dice) look different from print(hand)?

A list shows its items with Die's __repr__. print(hand) uses the Hand's own __str__.

Can I sort the dice to find straights?

Not with hand.dice.sort(): that moves the player's dice. sorted(hand.values()) makes a new list. has_run doesn't need sorting at all.

Why None and not 0 for an empty category?

0 is a real score: you can choose to take a 0. None means the category hasn't been used yet.